cm93depth_pi

Building the OpenCPN plugin on Linux and Raspberry Pi

Files, directories and procedure

Plugin version 1.5.1 | OpenCPN 5.8 and later

Linux Mint, Ubuntu, Debian (x86_64) | Raspberry Pi OS (arm64, armhf)

Contents

1 Overview **2**
1.1 Why the build must match the machine 2
2 Requirements **2**
2.1 Supported systems 2
2.2 Build tools and libraries 3
3 The source package **3**
3.1 Directory layout 3
3.2 Files used by a Linux build 3
3.3 Source files 4
4 Procedure - Linux Mint, Ubuntu, Debian **4**
5 Procedure - Raspberry Pi **5**
6 What the build script does **5**
6.1 Options 6
6.2 The plugin tarball 6
7 Building by hand (without the script) **6**
8 Optional: unit tests **6**
9 Updating, other machines, cleaning up **7**
10 Troubleshooting **7**

1 Overview

The plugin is built on the machine it is for (or one exactly like it) and packaged as a plugin tarball, which OpenCPN then installs with Options > Plugins > Import plugin.... One build serves every PC with the same distribution release and processor architecture.

Step	Where	Result
1. Prepare	Each type of machine: one Mint PC, one Pi	OpenCPN 5.8+ installed, source unzipped
2. Build	Same machine	`linux/dist/cm93depth_pi-<version>-0_<distro>-<release>-<arch>.tar.gz`
3. Install	Any machine of that type	Plugin imported and enabled in OpenCPN

A build takes about a minute on a PC and a few minutes on a Raspberry Pi 4 or 5. Internet access is needed once, for the build tools.

1.1 Why the build must match the machine

- A Linux plugin is a shared library (`libcm93depth_pi.so`) loaded into OpenCPN. It must use the same wxWidgets library build as that OpenCPN, and the same processor architecture.
- Building on the target machine against the distribution's own wxWidgets development package guarantees this. A library built on another distribution release may load and then crash, or not load at all.
- x86_64 (PCs), arm64 (64-bit Raspberry Pi OS) and armhf (32-bit Raspberry Pi OS) each need their own build.

2 Requirements

2.1 Supported systems

System	OpenCPN	Notes
Linux Mint 21 / 22, Ubuntu 22.04 / 24.04	5.8 or later from the OpenCPN PPA (`ppa:opencpn/opencpn`) or the distribution if 5.8+	x86_64.
Debian 13 'trixie'	`apt install opencpn` (5.10)	x86_64, arm64.

System	OpenCPN	Notes
Raspberry Pi OS based on trixie	`apt install opencpn` (5.10)	Pi 4 or 5 recommended; 64-bit (arm64) or 32-bit (armhf).
Raspberry Pi OS / Debian 12 'bookworm'	5.6 - too old	Move to the trixie-based release.
Flatpak OpenCPN	-	Not supported by this procedure (needs a Flatpak SDK build).

Check the OpenCPN version with Help > About, or `dpkg -l opencpn` in a terminal.

2.2 Build tools and libraries

The build script installs these with apt if they are missing (it asks for your password once):

Package	Provides
build-essential	g++, make, the C/C++ libraries headers
cmake	The build system (CMakeLists.txt)
libwxgtk3.2-dev (else libwxgtk3.0-gtk3-dev)	wxWidgets headers and wx-config: the same library OpenCPN uses
liblzma-dev	Reading compressed (.xz) CM93 cells
libgl-dev	OpenGL headers: drawing the look-ahead zone when OpenCPN runs in OpenGL mode
lsb-release	Distribution name and release, used in the package name

Disk space: about 300 MB for the tools and libraries, 50 MB for the build.

3 The source package

Everything is in **cm93depth_pi-<version>-src.zip** - the same package used for Windows. Unzip it into your home folder; this gives `~/cm93depth_pi`.

3.1 Directory layout

```
cm93depth_pi/
 CMakeLists.txt          build description (CMake)
 COPYING                 licence (GPL v2 or later)
 README.md               overview, sentence formats, build notes
 api/
   ocpn_plugin.h         OpenCPN plugin interface header (from OpenCPN 5.14.2)
 src/                    plugin source code
 linux/
   build_linux.sh        build and package script for Linux and Raspberry Pi
   README-linux.txt      short version of this document
   work/                 created by the build: build tree, staging (can be deleted)
   dist/                 created by the build: the plugin tarball(s)
 windows/                Windows build (not used on Linux)
 tests/                  unit tests (optional)
 doc/                    manual
```

3.2 Files used by a Linux build

File / directory	Role	Needed
`CMakeLists.txt`	Lists the sources, finds wxWidgets, liblzma and OpenGL, sets compiler options, builds `libcm93depth_pi.so`	Yes
`api/ocpn_plugin.h`	OpenCPN plugin interface. The bundled 5.14.2 header builds plugins for every OpenCPN from 5.8; the plugin uses only API 1.18	Yes
`src/*.cpp, src/*.h`	Plugin code (table 3.3)	Yes
`linux/build_linux.sh`	Installs tools, builds, packages	Recommended

File / directory	Role	Needed
`tests/`	Unit tests of the CM93 and S-57 readers, depth engine, look-ahead, sentences, alarm logic	Optional
`windows/`, `doc/`, `README.md`	Not used by the build	No

3.3 Source files

File	Content
`cm93_reader.cpp/.h`	CM93 cell decoder: dictionary, cell files, edges, soundings, attributes
`s57_reader.cpp/.h`	S-57 ENC reader: ISO 8211 records, features, geometry, updates
`s57_codes.cpp`	S-57 object and attribute catalogue (generated from OpenCPN's s57data)
`enc_index.cpp/.h`	Finds and indexes the ENC cells in the chart folders, with a cache file
`depth_engine.cpp/.h`	Depth at the vessel: chart band selection (CM93 and ENC), search modes, interpolation, land, gaps, dangers; diagnostics report
`lookahead.cpp`	Look-ahead zone, scale per slice, exact clipping, first shoal, gaps
`nmea_out.cpp/.h`	$PCMLD, $PCMLA and $IIALR sentences, checksum
`serial_port.cpp/.h`	Serial port output (POSIX on Linux)
`alarm_logic.h`	Grounding alarm states and evaluation
`alarm_dialog.cpp/.h`	Alarm window
`cm93depth_pi.cpp/.h`	Plugin entry, USER SETTINGS block (defaults), timer, output, chart overlay, config
`depth_panel.cpp/.h`	Display window
`prefs_dialog.cpp/.h`	Preferences dialog
`icons.cpp/.h`	Toolbar icon

The first eight (plus alarm_logic.h) contain no wxWidgets code; the rest is the OpenCPN / wxWidgets user interface.

4 Procedure - Linux Mint, Ubuntu, Debian

Step 1 Install OpenCPN 5.8 or later (if not already)

```
sudo add-apt-repository ppa:opencpn/opencpn     # Mint / Ubuntu only
sudo apt update
sudo apt install opencpn
```

Start OpenCPN once and close it, so its configuration exists.

Step 2 Unzip the source

```
cd ~
unzip ~/Downloads/cm93depth_pi-1.5.1-src.zip      # gives ~/cm93depth_pi
```

Step 3 Build and package

```
bash ~/cm93depth_pi/linux/build_linux.sh
```

The script prints each stage ('==> Looking for OpenCPN', 'Checking build tools', 'Building', 'Packaging the plugin for OpenCPN') and ends with:

```
Built and packaged:
   /home/<you>/cm93depth_pi/linux/dist/cm93depth_pi-1.5.1-0_linuxmint-22-x86_64.tar.gz
To install: start OpenCPN, Options > Plugins > Import plugin..., choose this
file, then enable 'CM93 Least Depth' in the plugin list.
```

Step 4 Install in OpenCPN

- Start OpenCPN. Options > Plugins > **Import plugin...**, choose the `.tar.gz` from `~/cm93depth_pi/linux/dist/`.
- OpenCPN reports 'Plugin CM93 Least Depth successfully imported'.
- Find **CM93 Least Depth** in the list and enable it. The toolbar button appears.

Step 5 Serial ports (if the plugin drives one)

```
sudo usermod -aG dialout $USER     # then log out and back in
ls /dev/ttyUSB* /dev/ttyACM*        # USB serial adapters
```

5 Procedure - Raspberry Pi

Step 1 Raspberry Pi OS and OpenCPN

- Use Raspberry Pi OS based on Debian 13 'trixie' (64-bit recommended). Check with `cat /etc/os-release`.
- A Pi 4 (4 GB or more) or Pi 5, with a fan or heat sink for the build.

```
sudo apt update
sudo apt install opencpn
```

Start OpenCPN once and close it.

Steps 2 - 4 As for Mint

```
cd ~
unzip cm93depth_pi-1.5.1-src.zip
bash ~/cm93depth_pi/linux/build_linux.sh
```

Result, e.g. `~/cm93depth_pi/linux/dist/cm93depth_pi-1.5.1-0_debian-13-arm64.tar.gz` (`armhf` on 32-bit Pi OS). Import it in OpenCPN and enable the plugin.

Step 5 Serial and sound on the Pi

- The Pi's own UART is `/dev/serial0` (enable it with `sudo raspi-config` > Interface Options > Serial Port: login shell No, hardware Yes). USB adapters are `/dev/ttyUSB0`. Add your user to `dialout` as above.
- Choose the audio output (HDMI, headphone, USB) in the desktop's volume menu, then use Test on the plugin's Alarm tab.

6 What the build script does

`linux/build_linux.sh`, in order:

Stage	Action
1. OpenCPN	Finds `opencpn` and its package version; stops if it is older than 5.8, or if only the Flatpak OpenCPN is installed.
2. Tools	Installs any missing package of section 2.2 with apt (sudo).
3. Build	Fresh build tree in `linux/work/build`; `cmake -DCMAKE_BUILD_TYPE=Release -DOCPN_API_DIR=<source>/api`; compiles on all processor cores; strips the library.
4. Package	Reads the version from `src/cm93depth_pi.h`, the distribution and release (lsb_release) and architecture (uname -m); writes `metadata.xml`; packs `<top>/metadata.xml` and `<top>/lib/opencpn/libcm93depth_pi.so` into `linux/dist/<top>.tar.gz`.
5. Install (only with --install)	Copies the library to `~/.local/lib/opencpn/` and sets `bEnabled=1` under `[PlugIns/libcm93depth_pi.so]` in `~/.opencpn/opencpn.conf` (backup `opencpn.conf.bak-cm93depth`).

6.1 Options

Option	Effect
(none)	Build and package. Nothing in OpenCPN is changed.
`--install`	Also install for the current user directly (development and testing).
`-h`, `--help`	Show the usage text.

6.2 The plugin tarball

```
cm93depth_pi-1.5.1-0_debian-13-arm64/
 metadata.xml                      name, version, API 1.18, target, architecture
 lib/opencpn/libcm93depth_pi.so    the plugin
```

OpenCPN's importer installs `lib/opencpn/libcm93depth_pi.so` to `~/.local/lib/opencpn/` for the current user and registers it so it can be uninstalled from the plugin list. The metadata target is `all` (accepted by OpenCPN's import check on any platform); the file name says which system it was built for.

7 Building by hand (without the script)

The same result step by step, e.g. on a system where you want to see each command:

```
# 1. tools
sudo apt install build-essential cmake lsb-release libwxgtk3.2-dev liblzma-dev libgl-dev

# 2. configure (out-of-source build directory)
cmake -S ~/cm93depth_pi -B ~/cm93build \
     -DCMAKE_BUILD_TYPE=Release -DOCPN_API_DIR=$HOME/cm93depth_pi/api

# 3. compile
cmake --build ~/cm93build -j$(nproc)

# 4a. install directly for this user ...
mkdir -p ~/.local/lib/opencpn
cp ~/cm93build/libcm93depth_pi.so ~/.local/lib/opencpn/
#     ... then start OpenCPN and enable CM93 Least Depth in Options > Plugins

# 4b. ... or package it for Import plugin (layout as in 6.2)
mkdir -p ~/pkg/cm93depth_pi-local/lib/opencpn
cp ~/cm93build/libcm93depth_pi.so ~/pkg/cm93depth_pi-local/lib/opencpn/
#     write ~/pkg/cm93depth_pi-local/metadata.xml (copy one from a script-built tarball)
tar -czf ~/cm93depth_pi-local.tar.gz -C ~/pkg --no-recursion \
   cm93depth_pi-local/metadata.xml cm93depth_pi-local/lib/opencpn/libcm93depth_pi.so
```

CMake reports during configure whether .xz cells are supported (liblzma) and whether the OpenGL zone drawing is built (OpenGL found); without OpenGL the zone is drawn only in non-OpenGL chart mode.

8 Optional: unit tests

The tests exercise the chart reader, depth engine, look-ahead, sentences and alarm logic on synthetic CM93 cells. They need no OpenCPN.

```
cmake -S ~/cm93depth_pi -B ~/cm93test -DOCPN_API_DIR=$HOME/cm93depth_pi/api \
     -DCM93DEPTH_BUILD_TESTS=ON
cmake --build ~/cm93test -j$(nproc) --target test_core test_serial
~/cm93test/test_core        # ends with: <n> passed, 0 failed
~/cm93test/test_serial
```

9 Updating, other machines, cleaning up

- **New plugin version**: unzip the new source (replacing or beside the old folder), run the script again, import the new tarball - it replaces the old plugin.
- **Other machines of the same type** (same distribution release and architecture): copy the tarball and import it; no build tools needed there.
- **OpenCPN upgraded**: if the new OpenCPN uses a different wxWidgets, or the distribution release changes (e.g. Mint 21 to 22), build again.
- **Cleaning up**: `linux/work/` can be deleted after a build. The build tools can be removed with `sudo apt remove libwxgtk3.2-dev` etc. (keep them for future builds).
- **Uninstalling**: Options > Plugins > CM93 Least Depth > Uninstall (or delete `~/.local/lib/opencpn/libcm93depth_pi.so` with OpenCPN closed).

10 Troubleshooting

Message / symptom	Cause and remedy
'OpenCPN ... is too old'	Install OpenCPN 5.8+ (section 2.1).
'Only the Flatpak OpenCPN is installed'	Install OpenCPN with apt / the PPA instead.
apt cannot find libwxgtk3.2-dev	Ubuntu 22.04 / Mint 21: add the OpenCPN PPA first (it carries wxWidgets 3.2 for OpenCPN), then run the script again.
CMake: 'Could NOT find wxWidgets'	The wx development package is missing or `wx-config` is not on PATH: `sudo apt install libwxgtk3.2-dev`.
Compiler killed / out of memory on a Pi	Close other programs, or build with one job: edit `-j"$(nproc)"` to `-j1` in build_linux.sh.
Plugin not in the list after import	Built on a different release or architecture. Look in `~/.opencpn/opencpn.log` for lines with 'cm93depth'; build on this type of machine.
OpenCPN log: 'incompatible' or 'undefined symbol'	Built against a different wxWidgets or OpenCPN older than 5.8. Rebuild on the target machine after updating OpenCPN.
Serial port 'Permission denied'	Add the user to `dialout` and log in again; or send through an OpenCPN connection.
No alarm sound	Select the audio output; Test on the Alarm tab.

If a build fails, the last screen of the script's output (or the CMake error) identifies the cause.